HDOC-61

Two Sum Using Pointers and Nested Loops in C

Question / Problem Statement

Write a C program that reads the size and elements of a variable-length integer array nums[], containing only positive integers, and an integer target. Pass the array to a function named two_sum() using its address. Use pointers to pass the array between the calling function and two_sum().

- Find two distinct indices, i and j, such that nums[i] + nums[j] = target.
- Display the two indices separated by a space.
- Return the indices in any order.
- Display -1 if no such pair exists.
- Use a nested-loop approach to solve the problem.
- Assume that at most one valid solution exists.

Algorithm / Logic

1. Read the number of elements n.

2. Declare a variable-length array nums[n] and read its positive integer elements.

3. Read the target value.

4. Pass the address of the array, its size, and target to two_sum().

5. Inside two_sum(), use an outer loop for index i from 0 to n - 2.

6. Use an inner loop for index j from i + 1 to n - 1 so that two different elements are compared.

7. Access elements through pointer arithmetic: *(nums + i) and *(nums + j).

8. If their sum equals target, display i and j and return immediately.

9. If all pairs are checked and no match is found, display -1.

C Program

```
#include <stdio.h>

void two_sum(int *nums, int n, int target)
{
    int i, j;

    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (*(nums + i) + *(nums + j) == target)
            {
                printf("%d %d\n", i, j);
                return;
            }
```

```
        }
    }

    printf("-1\n");
}

int main(void)
{
    int n, target, i;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid array size.\n");
        return 0;
    }

    int nums[n];

    printf("Enter %d positive integers: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", nums + i);

    printf("Enter target: ");
    scanf("%d", &target);

    printf("Indices: ");
    two_sum(nums, n, target);

    return 0;
}
```

Explanation

The array name nums represents the address of its first element. In main(), nums is passed to two_sum(), whose parameter int *nums receives this address. The nested loops examine every unique pair of indices. The expression *(nums + i) is equivalent to nums[i], so the program explicitly demonstrates pointer-based array access. The inner loop starts at i + 1, which guarantees that the same element is not used twice. When a pair sums to target, the indices are printed and the function returns. If no pair matches, -1 is printed.

Function Used

Function	Purpose	Parameters
two_sum()	Searches for two distinct array elements whose sum equals target.	int *nums, int n, int target

Header File Used

#include <stdio.h> - used for printf() and scanf().

Test Cases

Test	Array	Target	Expected Output	Reason
1	[2, 7, 11, 15]	9	0 1	2 + 7 = 9
2	[3, 2, 4]	6	1 2	2 + 4 = 6
3	[1, 5, 8, 12]	20	2 3	8 + 12 = 20
4	[1, 2, 3, 4]	10	-1	No pair sums to 10

Sample Output

```
Enter the size of array: 4
Enter 4 positive integers: 2 7 11 15
Enter target: 9
Indices: 0 1
```

Result

Thus, the C program successfully uses pointers and nested loops to find two distinct indices whose corresponding array elements add up to the given target. It displays the valid indices when a solution exists and -1 when no valid pair exists.